

MODULE 2

Unit Test Frameworks

Name	Chirashree Pan
Program / Year	B.Tech CSE (IoT, CS & BT) / 4th Year
Institution	Institute of Engineering & Management, Kolkata
Enrollment No	12023002029030
Experiments	Experiments 2.1 – 2.3
Date	17th September

Index — Module 2 Experiments

Exp. No.	Experiment Title	Status
2.1	UnitTest – First Test Case, Setup/Teardown, Assertions and Test Suite	Completed
2.2	PyTest Framework – Fixtures, Assertions, Multiple Tests and HTML Reporting	Completed
2.3	Page Object Model – Page Classes, Base Class, Test Class and CSV Data	Completed

Experiment 2.1 : unittest Framework – First Test Case, Setup/Teardown, Assertions and Test Suite

1. Problem Statement

The task is to create a basic Python unit test using the unittest framework. The test should open the SauceDemo login page, verify the page title, perform a valid login, use assertions to check the expected result, and close the browser after each test. The experiment also demonstrates setup, teardown, and running multiple test methods as a test suite.

2. Objective

The aim is to understand the structure of Python's unittest framework and practise test cases, assertions, setup and teardown methods, and a test suite using Selenium WebDriver.

3. Tools, Software & Concepts Used

- Python 3.x
- Selenium 4
- unittest – Python's built-in unit testing framework
- Google Chrome browser with Selenium Manager
- VS Code / PyCharm IDE
- setUp() and tearDown() methods
- assertEquals() and assertIn() assertions
- unittest.TestSuite and TextTestRunner
- SauceDemo: <https://www.saucedemo.com/>

4. Source Code / Implementation Steps

```
import unittest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestSauceDemo(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.get("https://www.saucedemo.com/")
        time.sleep(3)
        print("Browser opened successfully")

    def test_title(self):
        self.assertEqual(self.driver.title, "Swag Labs")
        print("Title test passed")
        time.sleep(2)

    def test_login(self):
        self.driver.find_element(By.ID, "user-name").send_keys("standard_user")
        self.driver.find_element(By.NAME, "password").send_keys("secret_sauce")
        self.driver.find_element(By.ID, "login-button").click()
        time.sleep(3)

        self.assertIn("inventory.html", self.driver.current_url)
        print("Login test passed")
        print("Current URL:", self.driver.current_url)
        time.sleep(3)

    def tearDown(self):
```

```

        time.sleep(2)
        self.driver.quit()
        print("Browser closed")

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

5. Output

5.1 TERMINAL OUTPUT

```

PS C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport Module2> python Experiment_2.1.py
test_login (__main__.TestSauceDemo.test_login) ... Browser opened successfully
Login test passed
Current URL: https://www.saucedemo.com/inventory.html
Browser closed
ok
test_title (__main__.TestSauceDemo.test_title) ... Browser opened successfully
Title test passed
Browser Closed
ok
-----
Ran 2 tests in 30.997s

OK
PS C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport Module2>

```

Fig 2.1.1: VS Code terminal output showing the test execution and result.

5.2 SELENIUM / TEST OUTPUT

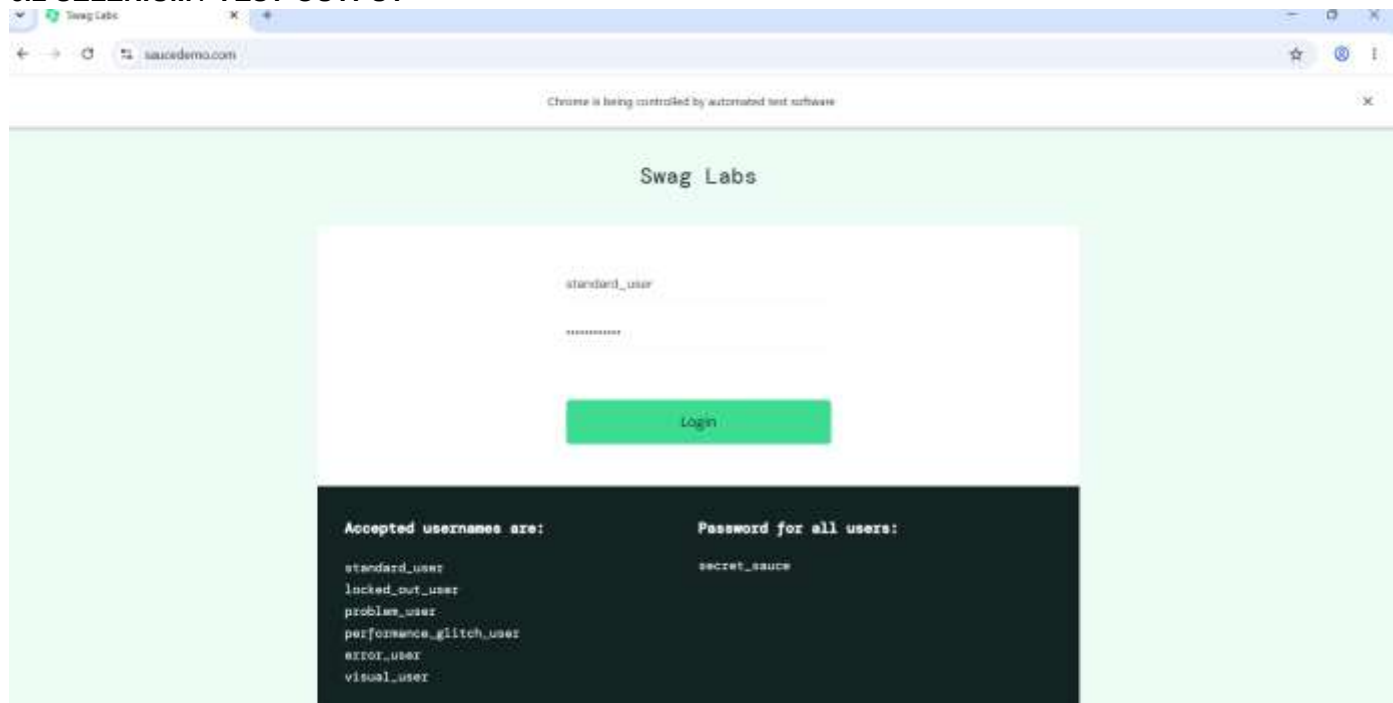


Fig 2.1.2: Selenium browser Showing logging

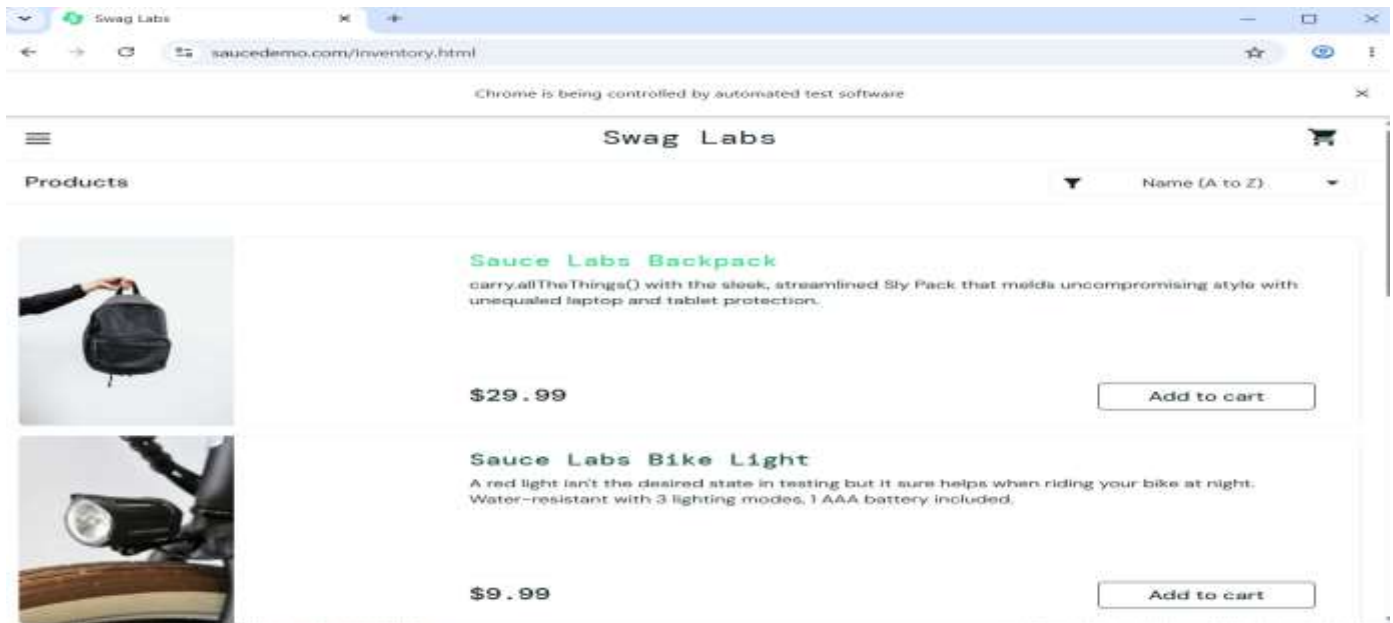


Fig 2.1.3: SauceDemo page / successful login visible in Chrome.

6. Result, Observation & Conclusion

Result: The unittest program executed the page-title and valid-login test cases successfully.

Observation: The VS Code terminal displayed the individual test results and success messages, while the Selenium browser showed the SauceDemo page and successful login. The `setUp()` method opened the browser before each test and `tearDown()` closed it after each test.

Conclusion: The terminal and Selenium outputs confirmed that the unittest framework worked correctly with Selenium. I learned how test methods, assertions, `setUp()`, `tearDown()`, `TestSuite` and `TextTestRunner` can be combined to organise automated tests.

Experiment 2.2 : PyTest Framework – Fixtures, Assertions, Multiple Tests and HTML Reporting

1. Problem Statement

The task is to run Selenium tests using the PyTest framework. The script should use a fixture for browser initialization and cleanup, perform a page-title test and a valid-login test, and run the test file from the VS Code terminal. An HTML report can also be generated to record the execution results.

2. Objective

The aim is to understand PyTest test-file and test-function naming conventions, fixtures, assertions, multiple test cases and HTML reporting for Selenium automation.

3. Tools, Software & Concepts Used

- Python 3.x
- Selenium 4
- pytest
- pytest-html plugin for HTML reports
- Google Chrome browser with Selenium Manager
- VS Code terminal
- PyTest fixture for driver setup and teardown
- assert statements for validation
- SauceDemo: <https://www.saucedemo.com/>

4. Source Code / Implementation Steps

```
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.get("https://www.saucedemo.com/")
    time.sleep(3)
    print("Browser opened")
    yield driver
    time.sleep(2)
    driver.quit()
    print("Browser closed")

def test_title(driver):
    assert driver.title == "Swag Labs"
    print("Title test passed")
    time.sleep(2)

def test_login(driver):
    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()
    time.sleep(3)

    assert "inventory.html" in driver.current_url
    print("Login successful")
    print("Current URL:", driver.current_url)
    time.sleep(3)
```

Run from VS Code terminal: `pytest -v Experiment_2_2.py`

Generate HTML report: `pytest -v Experiment_2_2.py --html=report.html`

5. Output

5.1 TERMINAL OUTPUT

```
(venv) PS C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport Module2> pytest -v Experiment_2_2.py -s
=====test session starts=====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport\
t Module2\.venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.22620-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy':
'1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport Module2
plugins: html-4.2.0, metadata-3.1.1
collected 2 items

Experiment_2_2.py::test_title Browser opened
Title test passed
PASSED Browser closed

Experiment_2_2.py::test_login Browser opened
Login successful
Current URL: https://www.saucedemo.com/inventory.html
PASSED Browser closed

=====
(venv) PS C:\Users\CHIRASHREE PAN\OneDrive\ドキュメント\LabReport Module2> |
```

Fig 2.2.1: VS Code terminal output showing the test execution and result.

5.2 SELENIUM / TEST OUTPUT

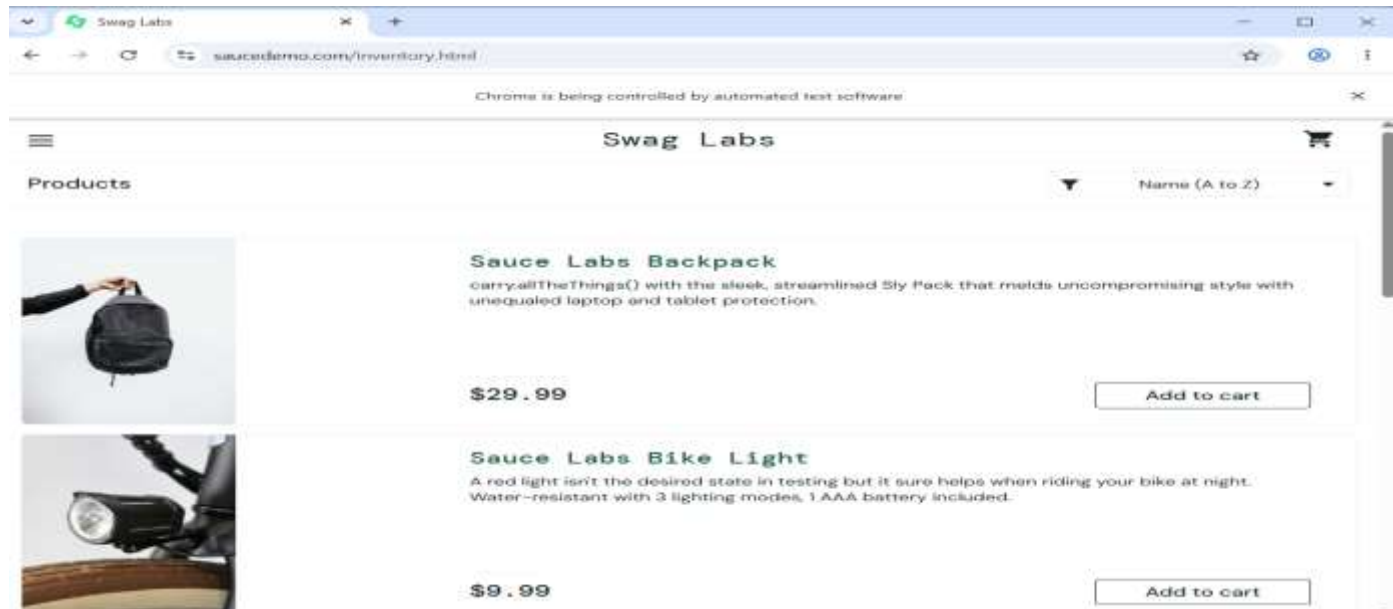


Fig 2.2.2: Selenium/browser output showing the completed automation step.

```

Experiment_2_2.py::test_title Browser opened
Title test passed
PASSEDBrowser closed

Experiment_2_2.py::test_login Browser opened
Login successful
Current URL: https://www.saucedemo.com/inventory.html
PASSEDBrowser closed

- Generated html report: file:///C:/Users/Shania%20Owais/OneDrive/%E3%83%89%E3%82%AD%E3%83%A5%E3%83%A1%E3%83%83%E3%83%88/LabReport%20Module2/report.html -

```

Fig 2.2.3: PyTest terminal showing collected tests and PASSED results.

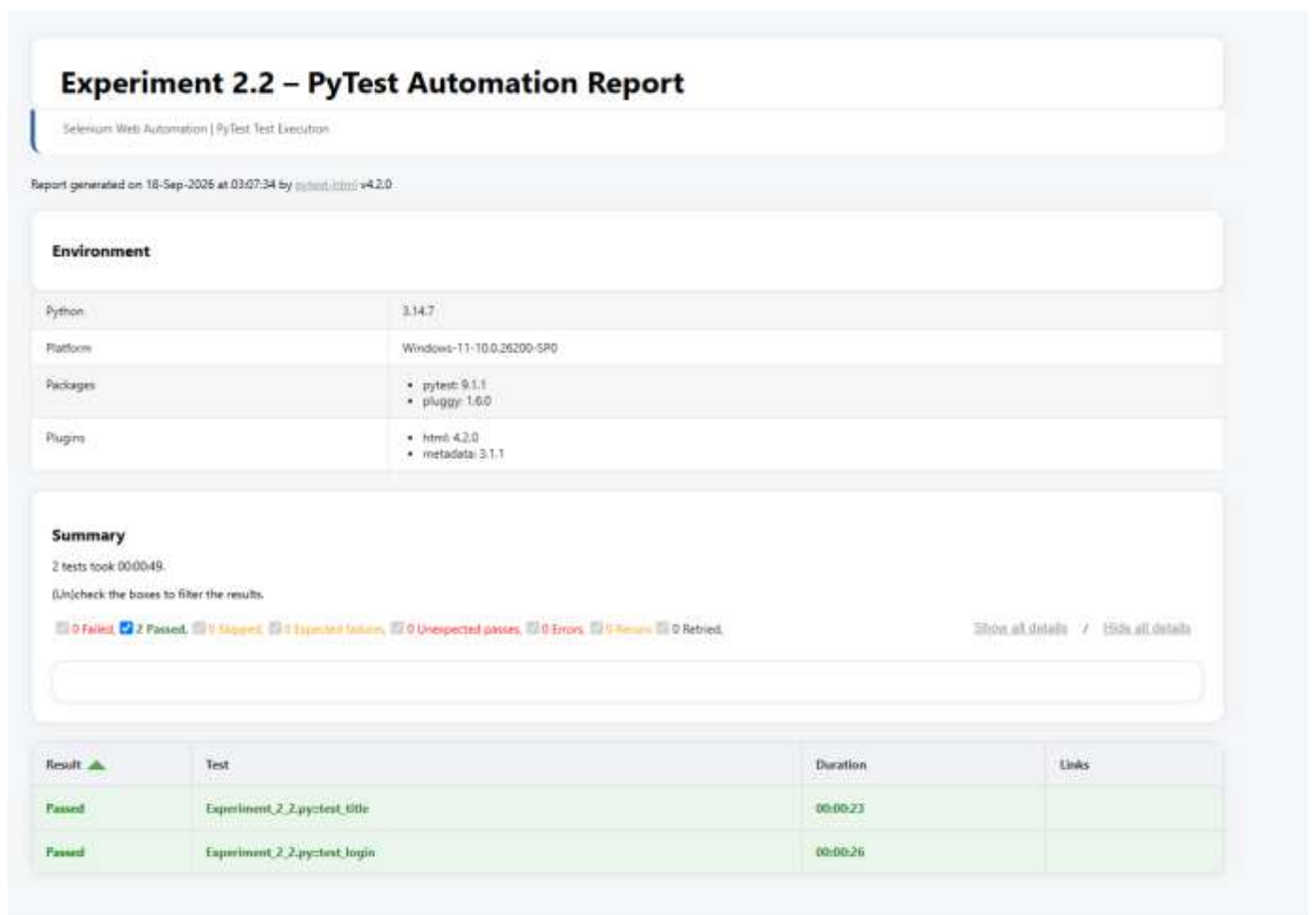


Fig 2.2.4: Generated report.html opened in the browser.

6. Result, Observation & Conclusion

Result: The Selenium tests were successfully executed through PyTest and the test results were displayed in the terminal. The HTML report was generated when the pytest-html command was used.

Observation: The terminal showed the test names and their PASSED status, while the Selenium browser showed the page and login actions. The fixture handled browser setup and cleanup automatically, so the test functions contained only the required test steps.

Conclusion: The terminal, Selenium and HTML-report outputs confirmed that PyTest can organise Selenium tests efficiently. I learned how fixtures reduce repeated setup code and how pytest-html can provide a readable execution report.

Experiment 2.3 : Page Object Model – Page Classes, Base Class, Test Class and CSV Data

1. Problem Statement

The task is to restructure a Selenium login test using the Page Object Model (POM). The locators and page actions should be kept inside page classes, while the test class should contain the actual test and assertions. A small CSV utility is also included to demonstrate reading test data separately from the test logic.

2. Objective

The aim is to understand the separation of base class, page class, test class and utility code and to practise a maintainable Selenium project structure using POM and external CSV test data.

3. Tools, Software & Concepts Used

- Python 3.x
- Selenium 4
- PyTest
- Google Chrome browser with Selenium Manager
- Page Object Model (POM)
- BasePage for common driver/page operations
- LoginPage for login locators and actions
- CSV module for external test data
- Assertions in the test class
- SauceDemo: <https://www.saucedemo.com/>

4. Source Code / Implementation Steps

A. testdata.csv

```
username,password
standard_user,secret_sauce
```

B. base_page.py

```
from selenium.webdriver.common.by import By

class BasePage:

    def __init__(self, driver):
        self.driver = driver

    def open(self, url):
        self.driver.get(url)

    def get_title(self):
        return self.driver.title
```

C. login_page.py

```
from selenium.webdriver.common.by import By
from base_page import BasePage

class LoginPage(BasePage):

    username = (By.ID, "user-name")
```

```

password = (By.ID, "password")
login_button = (By.ID, "login-button")

def enter_username(self, value):
    self.driver.find_element(*self.username).send_keys(value)

def enter_password(self, value):
    self.driver.find_element(*self.password).send_keys(value)

def click_login(self):
    self.driver.find_element(*self.login_button).click()

def login(self, username, password):
    self.enter_username(username)
    self.enter_password(password)
    self.click_login()

```

D. test_login.py

```

import csv
import time
from selenium import webdriver
from login_page import LoginPage

def read_data():
    with open("testdata.csv") as file:
        return list(csv.DictReader(file))

def test_login():

    data = read_data()[0]

    driver = webdriver.Chrome()
    driver.get("https://www.saucedemo.com/")
    time.sleep(3)

    page = LoginPage(driver)
    page.login(data["username"], data["password"])

    time.sleep(3)

    assert "inventory.html" in driver.current_url
    print("Login successful")
    print("Current URL:", driver.current_url)

    time.sleep(5)
    driver.quit()
    print("Browser closed")

```

Project structure:

```

Module2_POM/
|
|-- base_page.py
|-- login_page.py
|-- test_login.py
|-- testdata.csv

```

5. Output

5.1 TERMINAL OUTPUT

```
(.venv) PS C:\Users\Shania Owais\OneDrive\ドキュメント\LabReport Module2> pytest -v Experiment_2_3.py -s
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Shania Owais\OneDrive\ドキュメント\LabReport
Module2\.venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '
1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\Shania Owais\OneDrive\ドキュメント\LabReport Module2
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

Experiment_2_3.py/test_login.py::test_login Login successful
Current URL: https://www.saucedemo.com/inventory.html
Browser closed
PASSED
```

Fig 2.3.1: VS Code terminal output showing the test execution and result.

5.2 SELENIUM / TEST OUTPUT

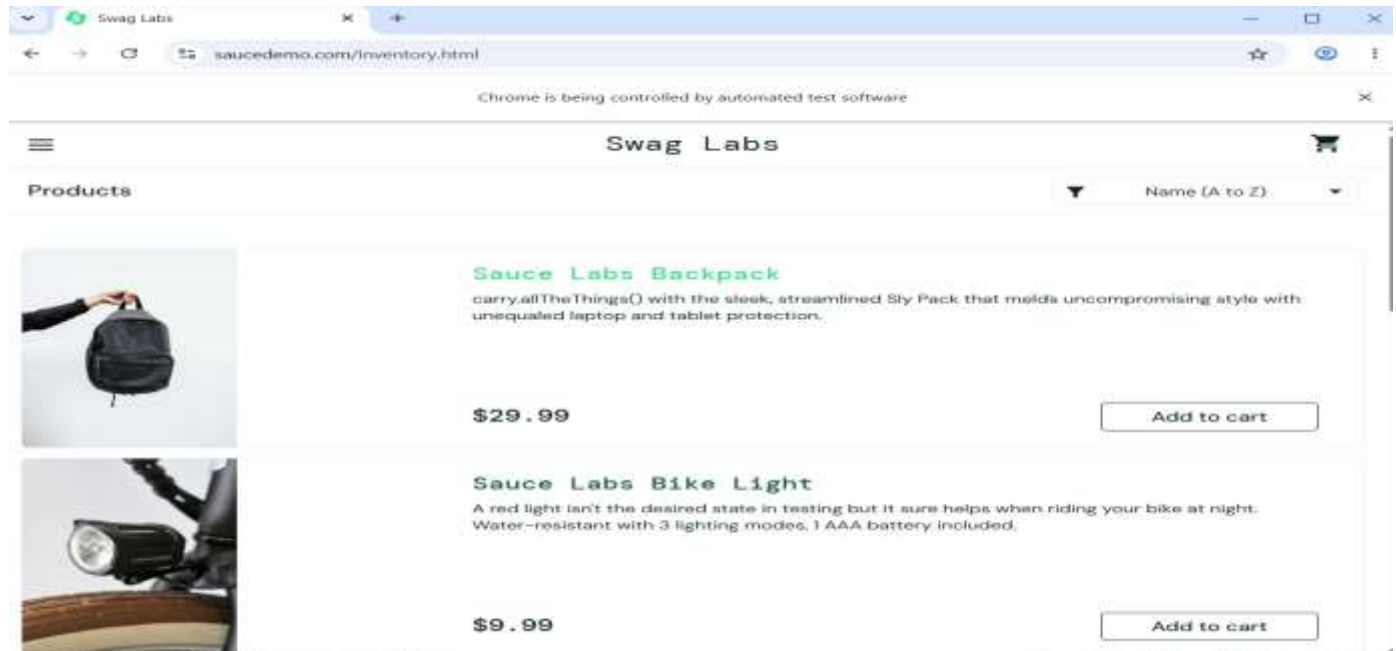


Fig 2.3.2 :Screenshot showing the SauceDemo Products page after successful login.

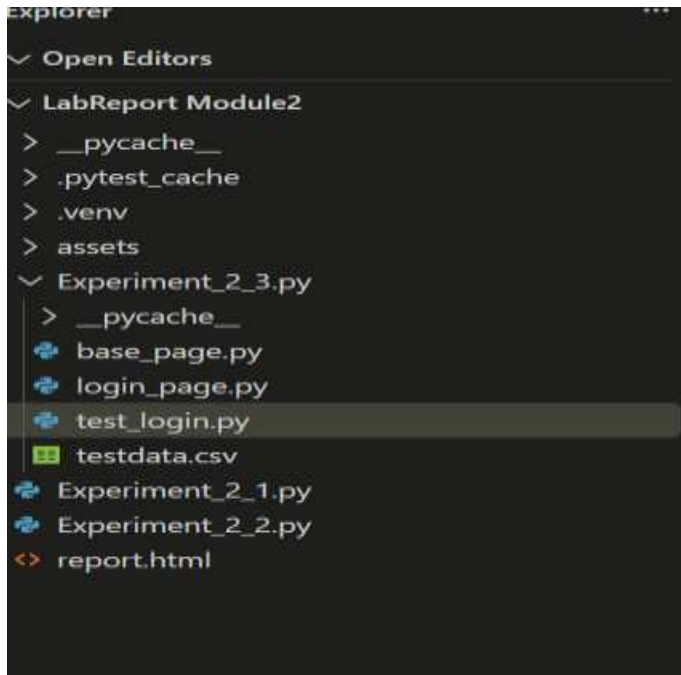


Fig 2.3.3: VS Code showing the POM project files.

6. Result, Observation & Conclusion

Result: The Selenium login test was successfully reorganised using the Page Object Model, and the login data was read from a separate CSV file.

Observation: The Selenium browser showed the SauceDemo login and inventory page, while the terminal confirmed that the POM login test passed. The page class contained the locators and UI methods, and the test class handled the test flow and assertion.

Conclusion: The terminal and Selenium outputs confirmed that the POM structure worked successfully. I learned how separating locators, page actions, test logic and test data makes Selenium automation easier to maintain and reuse.